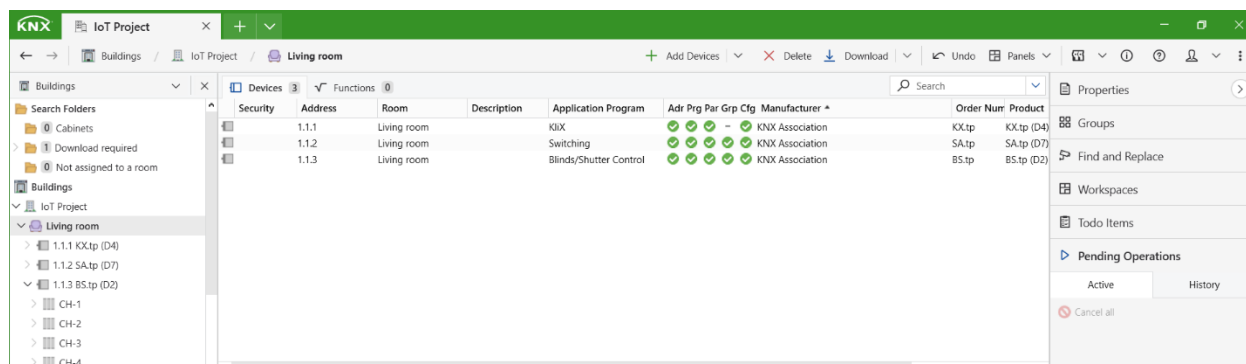
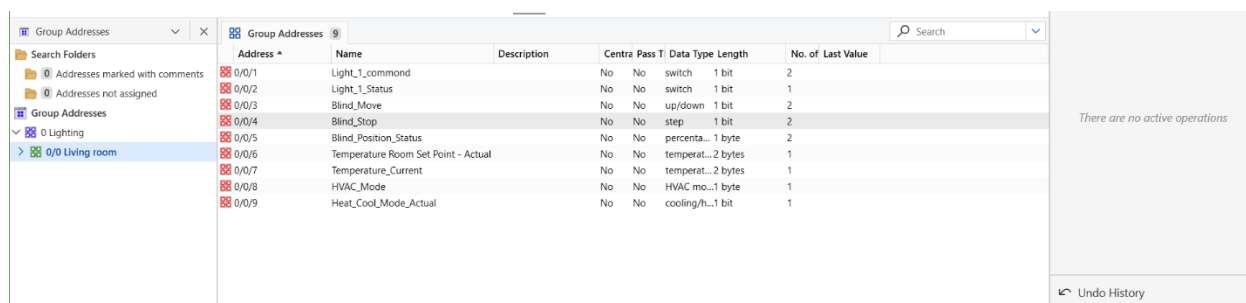


گزارش پروژه مبانی اینترنت اشیا

در این پروژه ابتدا با استفاده از ابزار *ETS6* یک ساختمان هوشمند طراحی شد. برای طراحی این ساختمان از دیوایس‌های *Klix* و ۲ عملگر استفاده شد. این ابزارها را در پایین می‌توانید مشاهده کنید:



برای پیکربندی این دستگاه‌ها و همچنین قرار دادن آن‌ها بر روی *KNX Virtual* نیاز به آدرس‌های گروهی بود. این آدرس‌ها را در پایین می‌توانید مشاهده کنید:



پس از پیکربندی دیوایس‌ها و قرار دادن آن‌ها بر روی ابزار شبیه‌ساز، وارد مرحله‌ی بعد می‌شویم یعنی بالا آوردن یک *gateway* برای برقراری ارتباط بین این ابزار شبیه‌ساز و نرم‌افزار *Thingsboard*. تنها نکته‌ی باقی‌مانده در این بخش قرار دادن مقدار *setpoint* بر روی شبیه‌ساز است. این مقدار قرار داده شد اما درست کار نکرد چون برای پیاده‌سازی آن نیاز به کامپوننت‌های دیگری بود و نحوه‌ی کارکرد این سیستم را به طور کلی تغییر می‌داد.

در مرحله‌ی بعد ابتدا یک *gateway* در نرم‌افزار *Thingsboard* درست کردیم اما نکته‌ی قابل توجه آن این بود که این دروازه هیچ راه ارتباطی‌ای با محیط بیرون از کانتینر خود را نداشت. به همین خاطر هیچ ارتباطی بین نرم‌افزار شبیه‌ساز و *Thingsboard* برقرار نمی‌شد. به همین خاطر مجبور به بیرون کشیدن کل دروازه شدیم و کل دروازه‌ی شبیه‌ساز را بر روی کامپیوتر خود به صورت لوکال اجرا کردیم. برای این کار ابتدا نیاز بود کل دروازه نرم‌افزار *Thingsboard* را بر روی یک فضای شبیه‌سازی شده‌ی پایتون دانلود کنیم. در مرحله‌ی بعد باید فایل مربوط به *gateway* خود را به درستی پیکربندی می‌کردیم. این بخش مهم‌ترین قسمت کل کار بود. زیرا در مراحل مختلف مقادیر مختلفی برای این فایل قرار داده شد ولی نرم‌افزار

Thingsboard پس از هر بار پیکربندی توانایی برقراری ارتباط با نرم افزار شبیه ساز را نداشت. در نهایت بعد از صرف مدت زمان خیلی زیادی، بالاخره به پیکربندی درست نرم افزار خود رسیدیم. در پایین می توانید این پیکربندی را مشاهده کنید:

```
"client": {
  "type": "TUNNELING",
  "addressFormat": "LONG",
  "localPort": 0,
  "gatewayIp": "127.0.0.1",
  "gatewayPort": 3671,
  "autoReconnect": true,
  "autoReconnectWait": 3,
  "gatewaysScanner": {
    "enabled": false
  }
}
```

```
"deviceInfo": {
  "deviceNameDataType": "string",
  "deviceNameExpressionSource": "constant",
  "deviceNameExpression": "KNX Smart Home",
  "deviceProfileDataType": "none",
  "deviceProfileExpressionSource": "constant",
  "deviceProfileNameExpression": "default"
},
```

همچنین در پایین می توانید کمی از لاگ هایی که این دروازه دارد را هم مشاهده کنید:

```
(.venv) PS C:\ThingsBoard\native-gateway> .\venv\Scripts\thingsboard-gateway.exe
2026-08-04 23:24:11.813 - [INFO] - [tb_gateway_service.py] - tb_gateway_service - __init__ - 155 - Gateway starting...
2026-08-04 23:24:11.881 - [INFO] - [tb_gateway_service.py] - tb_gateway_service - __init__ - 169 - ThingsBoard IoT gateway version: 3.8.3
2026-08-04 23:24:12.395 - [INFO] - [tb_loader.py] - tb_loader - import_module - 74 - Import KNXConnector from C:\ThingsBoard\native-gateway\venv\Lib\site-packages\thingsboard_gateway\connectors\knx.
2026-08-04 23:24:12.399 - [INFO] - [knx_connector.py] - knx_connector - __init__ - 79 - Starting KNX Connector...
2026-08-04 23:24:12.400 - [INFO] - [tb_gateway_service.py] - tb_gateway_service - __init_and_start_regular_connector - 1062 - Cannot find separated report strategy for connector 'my-knx-gateway'. The main report strategy will be used as a connector report strategy.
2026-08-04 23:24:12.483 - [INFO] - [tb_client.py] - tb_client - connect - 448 - Connecting to ThingsBoard...
2026-08-04 23:24:12.483 - [INFO] - [tb_client.py] - tb_client - __send_connect - 477 - Sending connect to 127.0.0.1:1883
2026-08-04 23:24:12.415 - [INFO] - [knx_connector.py] - knx_connector - __start_client_with_block - 155 - Connected to KNX bus
2026-08-04 23:24:12.441 - [INFO] - [tb_client.py] - tb_client - __on_connect - 368 - MQTT client connected to platform 127.0.0.1: 1883
2026-08-04 23:24:12.441 - [INFO] - [tb_device_mqtt.py] - tb_device_mqtt - __on_connect - 600 - MQTT client <paho.mqtt.client.Client object at 0x000001E02D8EE3C8> - Connected!
2026-08-04 23:24:12.456 - [INFO] - [tb_device_mqtt.py] - tb_device_mqtt - on_service_configuration - 927 - Service configuration was successfully retrieved and applied.
2026-08-04 23:24:12.457 - [INFO] - [tb_device_mqtt.py] - tb_device_mqtt - on_service_configuration - 928 - Current device limits: {'ratelimits': {'messages': None, 'telemetryMessages': None, 'telemetryDataPoints': None}, 'maxPayloadSize': 65536, 'maxInflightMessages': 100}
2026-08-04 23:24:12.457 - [INFO] - [tb_gateway_mqtt.py] - tb_gateway_mqtt - __on_service_configuration - 345 - Current limits for devices connected through the gateway: {'messages': None, 'telemetryMessages': None, 'telemetryDataPoints': None}
2026-08-04 23:24:14.374 - [INFO] - [tb_gateway_service.py] - tb_gateway_service - __init__ - 242 - Gateway core started.
2026-08-04 23:24:14.375 - [INFO] - [tb_gateway_mqtt.py] - tb_gateway_mqtt - gw_subscribe_to_attribute - 267 - Subscribed to * with id 1 for device *
2026-08-04 23:24:14.376 - [INFO] - [tb_gateway_service.py] - tb_gateway_service - __init__ - 259 - Gateway connectors initialized.
2026-08-04 23:24:14.379 - [INFO] - [statistics_service.py] - statistics_service - __send_statistics - 234 - Collected regular statistics: {'machineStats': {'totalCpuUsage': 0.0, 'diskUsage': 90.2, 'freeMemory': 95.0, 'freeDisk': '35.0G', 'gwProcessCpuUsage': 0.0, 'gwMemory': 0.5, 'gwProcessMemoryFullInfo': {'rss': 84586496, 'vms': 64528384, 'num_page_faults': 38078, 'peak_vms': 84586496, 'vset': 84586496, 'peak_paged_pool': 298232, 'paged_pool': 266248, 'peak_non_paged_pool': 145120, 'nonpaged_pool': 48872, 'pagefile': 64528384, 'peak_pagefile': 64536576, 'private': 64528384, 'uss': 58980480}, 'msgsSentToPlatform': 2, 'msgsReceivedFromPlatform': 0}, 'serviceStats': {'storageMsgPulled': 0, 'storageMsgCount': 1, 'platformMsgPushed': 0, 'platformAttrProduced': 0, 'platformTsProduced': 0}, 'connectorsStats': {'my-knx-gateway': {'storageMsgPushed': 1}}}
```

بعد از اینکه از ساخته شدن یک دیوایس *gateway* در *Thingsboard* مطمئن شدیم، مقادیر تستی را از *KNXVirtual* برای این دروازه ارسال کردیم. مقادیر به درستی برای این نرم‌افزار ارسال می‌شدند. با برقرار شدن این ارتباط، عملاً قسمت اصلی پروژه زده شده بود و فقط در مرحله‌ی بعدی باید دروازه را به گونه‌ای پیکربندی می‌کردیم که اطلاعات را بتواند به درستی دریافت کند و این اطلاعات برای این نرم‌افزار شبیه‌ساز معنادار باشد. درنهایت مجدداً بعد از صحیح و خطاهای متوالی و زیاد، به این پیکربندی رسیدیم که در پایین عکس آن قرار داده شده است:

```
1 {
2   "client": {
3     "type": "TUNNELING",
4     "addressFormat": "LONG",
5     "localPort": 0,
6     "gatewayIp": "127.0.0.1",
7     "gatewayPort": 3671,
8     "autoReconnect": true,
9     "autoReconnectWait": 3,
10    "gatewaysScanner": {
11      "enabled": false
12    }
13  },
14  "devices": [
15    {
16      "deviceInfo": {
17        "deviceNameDataType": "string",
18        "deviceNameExpressionSource": "constant",
19        "deviceNameExpression": "KNX Smart Home",
20        "deviceProfileDataType": "none",
21        "deviceProfileExpressionSource": "constant",
22        "deviceProfileNameExpression": "default"
23      },
24      "pollPeriod": 3000,
25      "attributes": [],
26      "timeseries": [
27        {
28          "type": "9.001",
29          "key": "roomTemperature",
30          "groupAddress": "0/0/7"
31        },
32        {
33          "type": "1.001",
34          "key": "lightState",
35          "groupAddress": "0/0/2"
36        },
37        {
38          "type": "5.001",
39          "key": "blindPosition",
40          "groupAddress": "0/0/5"
41        }
42      ]
43    },
44    "poolPeriod": 3000,
45    "attributeUpdates": [],
46    "serverSideRpc": [
47      {
48        "requestType": "write",
49        "deviceNameFilter": "^KNX Smart Home$",
50        "method": "setLight",
51        "dataType": "1.001",
52        "groupAddress": "0/0/1"
53      },
54      {
55        "requestType": "write",
56        "deviceNameFilter": "^KNX Smart Home$",
57        "method": "moveBlind",
58        "dataType": "1.008",
59        "groupAddress": "0/0/3"
60      },
61      {
62        "requestType": "write",
63        "deviceNameFilter": "^KNX Smart Home$",
64        "method": "stopBlind",
65        "dataType": "1.007",
66        "groupAddress": "0/0/4"
67      }
68    ],
69    "name": "my-knx-gateway",
70    "id": "d1c9cdf7-d49c-44e6-9afd-5d887f705818",
71    "logLevel": "DEBUG",
72    "enableRemoteLogging": false,
73    "configVersion": "3.8.3"
74  }
75 }
```

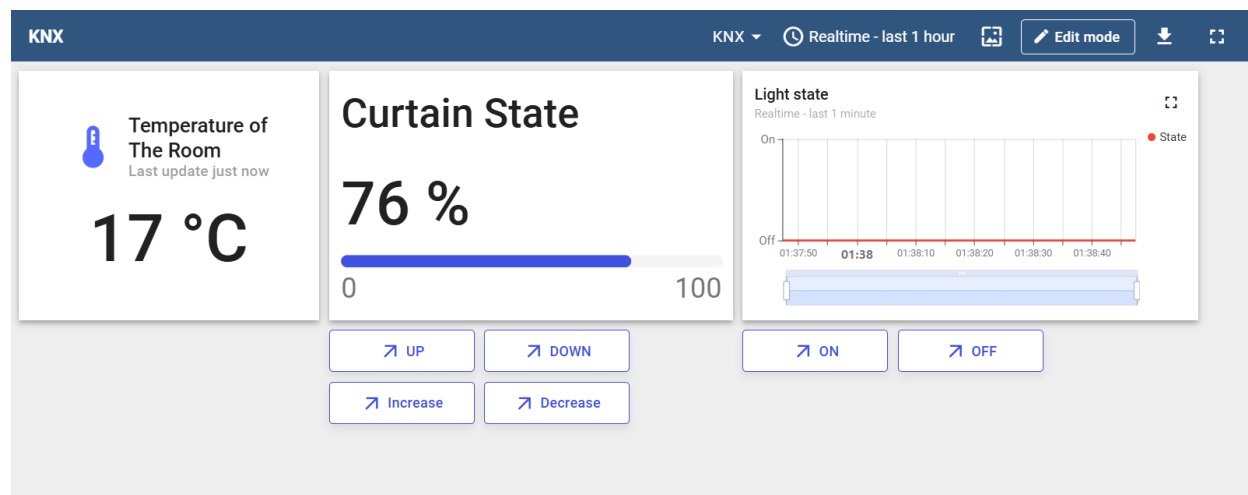
این پیکربندی هم شامل ارسال فرمان‌ها از طریق *RPC* می‌شود و هم دریافت مقادیر *Timeseries*. مهم‌ترین چالشی که در این قسمت برخوردیم بخش سویچ‌ها بود. در *Thingsboard* ما نوع داده‌ای با نام *Switch* نداریم و به همین خاطر مقادیری که از سویچ‌ها برای این نرم‌افزار فرستاده می‌شد عملاً دریافت نمی‌شد. به همین خاطر مجبور به تغییر در سورس‌کدهای اصلی *Thingsboard* شدیم. کدهای تغییر یافته در پایین را می‌توانید مشاهده کنید:

```

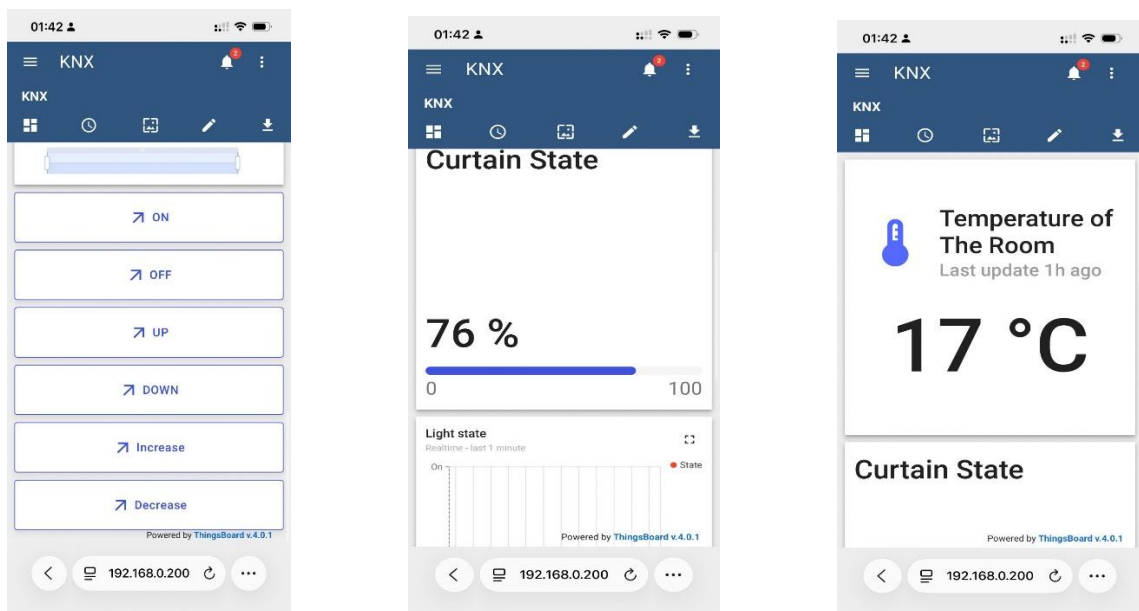
1 converted_value = data.get(config.get('groupAddress'))['response']
2
3         # Convert XKNX Enum values such as Switch.ON/OFF
4         # to JSON-compatible primitive values.
5         if hasattr(converted_value, 'value'):
6             converted_value = converted_value.value
7         elif hasattr(converted_value, 'as_dict'):
8             converted_value = converted_value.as_dict()
9
10        if converted_value is not None:
11            datapoint_key = TBUtility.convert_key_to_datapoint_key(config['key'],
12                                                                    device_report_strategy,
13                                                                    config,
14                                                                    self.__log)
15
16            if section == 'attributes':
17                converted_data.add_to_attributes({datapoint_key: converted_value})
18            elif section == 'timeseries':
19                converted_data.add_to_telemetry({datapoint_key: converted_value})

```

این کد در فایل *knx_uplink_converter.py* در آدرس *.venv\Lib\site-packages\thingsboard_g ateway\connectors\knx* قرار داشت. پس از پیاده‌سازی همه‌ی این موارد، به مرحله‌ی پیاده‌سازی داشبورد رسیدیم. برای داشبورد ۳ ویدجت برای نمایش وضعیت دما، پرده و چراغ قرار دادیم و همین‌طور ۶ دکمه برای روشن و خاموش کردن چراغ و بالا پایین کردن و متوقف کردن پرده. این داشبورد را در پایین می‌توانید مشاهده کنید:



برای مشخص کردن *setpoint* دکمه‌ای قرار داده نشد چون از ابتدا نحوه‌ی کار آن با چیزی که در صورت پروژه گفته شده بود متفاوت بود و به شدت ممکن بود دردسر ساز شود و به همین خاطر از زدن آن صرف نظر شد. هر کدام از این ۳ ویجت هر ۳ ثانیه یک بار آپدیت می‌شوند و می‌توانیم هر ۳ ثانیه وضعیت آن‌ها را مشاهده کنیم. در مرحله‌ی بعد هم سراغ اپلیکیشن موبایل رفتیم. در موبایل هم دقیقا همه‌ی این مواردی که در بالا هست قرار داده شده است. نکته‌ای که وجود دارد این است که به صورت مشخص از خود نرم‌افزار تینگزبورد استفاده نشده است و از نسخه‌ی تحت وب آن استفاده شده است. داشبوردهای مربوط به موبایل را هم در پایین می‌توانید مشاهده کنید:



با توجه به معماری‌ای که در پروژه استفاده شده بود، امکان استفاده از نسخه‌ی *cloud* نرم‌افزار *Thingsboard* و در نتیجه تغییرات به معنای واقعی کلمه *Remote* وجود نداشت.

بخش امتیازی

این بخش به نسبت بخش اصلی پروژه به مراتب راحت‌تر بود. برای اینکه این بخش را پیاده‌سازی کنیم ابتدا یک مدل *Ollama* را دانلود و روی سیستم خودمان نصب کردیم. در ادامه مشخصا مدل *qwen3:4b* را در پروژه قرار دادیم. در مرحله‌ی بعد یک *MCP Server* با استفاده از زبان *Type Script* نوشته شد تا بتوانیم قابلیت‌هایی که خانه‌ی هوشمند ما دارد را به صورت ابزار در اختیار مدل‌مان قرار دهیم. یک ابزار برای ارائه‌ی اطلاعات خانه و ابزار دیگر برای خاموش و روشن کردن چراغ خانه. به صورت کلی روندی که برای اجرای یک دستور طی می‌شود به شرح زیر است:

1. در کنسول متن فرمان نوشته می‌شود.

2. مدل متن را دریافت می‌کند.

3. مدل در نقش یک *Client* برای *MCPServer* عمل می‌کند و یک درخواست به سرور ارسال می‌کند با محتوای مربوط به فرمان.

4. درخواست در سرور پایش می‌شود و سپس به تینگزبورد ارسال می‌شود.

از این قسمت به بعد هم همانند قسمت اجباری مربوط به پروژه روند طی می‌شود. در پایین می‌توانید کدهای مربوط به *MCPServer Agent* را مشاهده کنید:

Agent.ts:

```
1 import "dotenv/config";
2
3 import { createInterface } from "node:readline/promises";
4 import { stdin as input, stdout as output } from "node:process";
5
6 import ollama from "ollama";
7 import { client } from "@modelcontextprotocol/sdk/client/index.js";
8 import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";
9
10 const MODEL = "qwen3:4b";
11
12 type McpTool = {
13   name: string;
14   description?: string;
15   inputSchema: Record<string, unknown>;
16 };
17
18 type OllamaToolCall = {
19   function: {
20     name: string;
21     arguments: Record<string, unknown>;
22   };
23 };
24
25 function mcpResultToText(result: unknown): string {
26   const value = result as {
27     content?: Array<{
28       type?: string;
29       text?: string;
30     }>;
31   };
32
33   const texts =
34     value?.content
35       ?.filter((item) => item.type === "text" && item.text)
36       .map((item) => item.text) ?? [];
37
38   return texts.length > 0 ? texts.join("\n") : JSON.stringify(result);
39 }
40
41 async function connectToMcp(): Promise<Client> {
42   const client = new Client({
43     name: "ollama-smart-home-client",
44     version: "1.0.0",
45   });
46
47   const transport = new StdioClientTransport({
48     command: process.platform === "win32" ? "corepack.cmd" : "corepack",
49     args: ["yarn", "tsx", "src/index.ts"],
50     cwd: process.cwd(),
51     env: process.env as Record<string, string>,
52   });
53
54   await client.connect(transport);
55
56   return client;
57 }
58
59 async function answer(
60   client: Client,
61   tools: McpTool[],
62   userMessage: string,
63 ): Promise<string> {
64   const ollamaTools = tools.map((tool) => ({
65     type: "function" as const,
66     function: {
67       name: tool.name,
68       description: tool.description ?? "",
69       parameters: tool.inputSchema,
70     },
71   }));
72
73   const messages: Array<Record<string, unknown>> = [
74     {
75       role: "system",
76       content:
77         "You are a Persian smart home assistant.\n\n" +
78         "Use the available tools whenever the user asks about the real state of the home\n" +
79         "or asks you to control a device.\n\n" +
80         "Never claim that a device was controlled unless the corresponding tool\n" +
81         "completed successfully.\n\n" +
82         "Answer the user in Persian.",
83     },
84     {
85       role: "user",
86       content: userMessage,
87     },
88   ];
89
90   for (let iteration = 0; iteration < 5; iteration += 1) {
91     const response = await ollama.chat({
92       model: MODEL,
93       messages: messages as never,
94       tools: ollamaTools,
95       stream: false,
96       think: false,
97     });
98   }
99 }
```

```

1 messages.push(response.message as unknown as Record<string, unknown>);
2
3 const toolCalls =
4   (response.message.tool_calls ?? []) as unknown as OllamaToolCall[];
5
6 if (toolCalls.length === 0) {
7   return response.message.content || "پاسخی از مدل دریافت نشد.";
8 }
9
10 for (const toolCall of toolCalls) {
11   const toolName = toolCall.function.name;
12   const toolArguments = toolCall.function.arguments ?? {};
13
14   console.log(
15     `\n[Tool] ${toolName}`,
16     JSON.stringify(toolArguments, null, 2),
17   );
18
19   try {
20     const result = await client.callTool({
21       name: toolName,
22       arguments: toolArguments,
23     });
24
25     const toolResult = mcpResultToText(result);
26
27     console.log(`[Result] ${toolResult}`);
28
29     messages.push({
30       role: "tool",
31       tool_name: toolName,
32       content: toolResult,
33     });
34   } catch (error) {
35     const message =
36       error instanceof Error ? error.message : String(error);
37
38     console.error(`[Tool error] ${message}`);
39
40     messages.push({
41       role: "tool",
42       tool_name: toolName,
43       content: `Tool failed: ${message}`,
44     });
45   }
46 }
47
48 throw new Error("Agent exceeded the maximum number of tool-call iterations.");
49 }
50
51 async function main(): Promise<void> {
52   const client = await connectToMcp();
53   const toolsResponse = await client.listTools();
54   const tools = toolsResponse.tools as McpTool[];
55
56   console.log(
57     `MCP connected. Tools: ${tools.map((tool) => tool.name).join(", ")}`,
58   );
59   console.log("برای خروج بنویس exit.\n");
60
61   const readline = createInterface({ input, output });
62
63   try {
64     while (true) {
65       const userMessage = (await readline.question("شما: ")).trim();
66
67       if (!userMessage) {
68         continue;
69       }
70
71       if (["exit", "quit", "خروج"].includes(userMessage.toLowerCase())) {
72         break;
73       }
74
75       try {
76         const response = await answer(client, tools, userMessage);
77         console.log(`دستیار: ${response}\n`);
78       } catch (error) {
79         console.error(
80           "\nخطا:",
81           error instanceof Error ? error.message : String(error),
82           "\n",
83         );
84       }
85     }
86   } finally {
87     readline.close();
88     await client.close();
89   }
90 }
91
92 main().catch((error: unknown) => {
93   console.error(
94     error instanceof Error ? error.stack ?? error.message : String(error),
95   );
96   process.exit(1);
97 });
98

```

*MCP*Server:


```

1 type LoginResponse = {
2   token: string;
3   refreshToken: string;
4 };
5
6 type TelemetryEntry = {
7   ts: number;
8   value: string;
9 };
10
11 type LatestTelemetry = Record<string, TelemetryEntry[]>;
12
13 const {
14   THINGSBOARD_URL,
15   THINGSBOARD_USERNAME,
16   THINGSBOARD_PASSWORD,
17   THINGSBOARD_DEVICE_ID,
18 } = process.env;
19
20 if (
21   !THINGSBOARD_URL ||
22   !THINGSBOARD_USERNAME ||
23   !THINGSBOARD_PASSWORD ||
24   !THINGSBOARD_DEVICE_ID
25 ) {
26   throw new Error("Missing ThingsBoard environment variables");
27 }
28
29 let cachedToken: string | null = null;
30
31 async function getAuthToken(): Promise<string> {
32   if (cachedToken) {
33     return cachedToken;
34   }
35
36   const response = await fetch(`${THINGSBOARD_URL}/api/auth/login`, {
37     method: "POST",
38     headers: {
39       "Content-Type": "application/json",
40     },
41     body: JSON.stringify({
42       username: THINGSBOARD_USERNAME,
43       password: THINGSBOARD_PASSWORD,
44     }),
45   });
46
47   if (!response.ok) {
48     throw new Error(
49       `ThingsBoard login failed: ${response.status} ${await response.text()}`,
50     );
51   }
52
53   const result = (await response.json()) as LoginResponse;
54   cachedToken = result.token;
55
56   return result.token;
57 }
58
59 async function tbFetch(
60   path: string,
61   init: RequestInit = {},
62 ): Promise<Response> {
63   const token = await getAuthToken();
64
65   const response = await fetch(`${THINGSBOARD_URL}${path}`, {
66     ...init,
67     headers: {
68       "Content-Type": "application/json",
69       "X-Authorization": `Bearer ${token}`,
70       ...init.headers,
71     },
72   });
73
74   if (response.status === 401) {
75     cachedToken = null;
76
77     const refreshedToken = await getAuthToken();
78
79     return fetch(`${THINGSBOARD_URL}${path}`, {
80       ...init,
81       headers: {
82         "Content-Type": "application/json",
83         "X-Authorization": `Bearer ${refreshedToken}`,
84         ...init.headers,
85       },
86     });
87   }
88
89   return response;
90 }

```

```

1 export async function getHomeStatus(): Promise<{
2   roomTemperature: number | null;
3   lightState: boolean | null;
4   blindPosition: number | null;
5 }> {
6   const keys = ["roomTemperature", "lightState", "blindPosition"].join(",");
7
8   const response = await tbFetch(
9     `/api/plugins/telemetry/DEVICE/${THINGSBOARD_DEVICE_ID}/values/timeseries?keys=${keys}`,
10  );
11
12  if (!response.ok) {
13    throw new Error(
14      `Reading telemetry failed: ${response.status} ${await response.text()}`,
15    );
16  }
17
18  const telemetry = (await response.json()) as LatestTelemetry;
19
20  const temperatureValue = telemetry.roomTemperature?.[0]?.value;
21  const lightValue = telemetry.lightState?.[0]?.value;
22  const blindValue = telemetry.blindPosition?.[0]?.value;
23
24  return {
25    roomTemperature:
26      temperatureValue !== undefined ? Number(temperatureValue) : null,
27
28    lightState:
29      lightValue !== undefined
30        ? ["true", "1", "on"].includes(lightValue.toLowerCase())
31        : null,
32
33    blindPosition:
34      blindValue !== undefined ? Number(blindValue) : null,
35  };
36 }
37
38 export async function setLight(state: boolean): Promise<void> {
39   const response = await tbFetch(
40     `/api/rpc/oneway/${THINGSBOARD_DEVICE_ID}`,
41     {
42       method: "POST",
43       body: JSON.stringify({
44         method: "setLight",
45         params: state,
46       }),
47     },
48   );
49
50   if (!response.ok) {
51     throw new Error(
52       `Light RPC failed: ${response.status} ${await response.text()}`,
53     );
54   }
55 }

```